**UNIVERSIDAD AUTÓNOMA DE CAMPECHE**

Facultad de Ingeniería

---

# Práctica de SQL Injection

## Detección, pruebas y corrección del login

**Seguridad Informática | Segundo parcial**

Alumno: Ulises Zarate Concha

Matrícula: 66575

Tecnologías: Node.js, Express y SQLite3

Fecha: 26 de octubre de 2026

Reporte académico de una aplicación local autorizada. Incluye payloads funcionales, explicación técnica, parametrización SQL, evidencias comparativas y Pull Request de la versión segura.

# Índice

1. Objetivo, entorno y metodología ..... 3

2. Hallazgos en la aplicación original ..... 4

3. Payloads que funcionaron y explicación ..... 5

4. Corrección mediante parametrización SQL ..... 6

5. Verificación y comparación de resultados ..... 7

6. Conclusiones y Pull Request ..... 8

Anexo: 11 casos y sus 22 capturas ..... 9

## Resumen de la entrega

Se ejecutaron 11 casos en cada versión mediante peticiones HTTP y se repitieron desde el formulario real del navegador. Se conservan 22 capturas de los formularios, una por ejecución.

Cuatro entradas de inyección distintas lograron que la versión vulnerable respondiera “Acceso concedido”. La versión corregida rechazó esas entradas y mantuvo el acceso con credenciales válidas.

Las capturas, los resultados y las explicaciones corresponden al proyecto de Ulises Zarate Concha. Las pruebas se limitaron a 127.0.0.1.

# 1. Objetivo, entorno y metodología

## Objetivo

Identificar cómo la interpolación de entradas modifica una consulta SQL de login, demostrar su efecto en un entorno local, corregir la consulta mediante parámetros y repetir las mismas pruebas para verificar el cambio.

## Entorno utilizado

Windows y PowerShell; Node.js 24.11.1; npm 11.3.0; Express 5.3.0 y sqlite3 6.0.1. La aplicación original se localizó en la carpeta 26-10-2026 del repositorio 66575_Zarate.

```
laboratorio-sqli/
  original/     Copia de los archivos iniciales
  vulnerable/   Servidor local en 127.0.0.1:3101
  corregida/    Servidor local en 127.0.0.1:3102
  evidencias/   Registros y capturas de las pruebas
P05-SQL-INJECTION/  Version segura incluida en el PR
```

## Procedimiento

1. Inspeccionar package.json, server.js, login.html y el esquema de SQLite en modo de solo lectura.

2. Conservar los originales y crear versiones y bases de datos separadas con credenciales ficticias.

3. Ejecutar controles positivos y negativos, tautologías, comentarios, UNION, inferencia booleana y una entrada con error de sintaxis.

4. Repetir las entradas mediante el formulario real, pulsar Ingresar, esperar la respuesta visible y guardar una captura por prueba.

5. Verificar la versión segura, documentar los resultados y crear el Pull Request.

Las dos bases de laboratorio se crearon por separado; no se alteró la base original. La cuenta de demostración es admin / 12345.

# 2. Hallazgos en la aplicación original

## 2.1 Consulta construida mediante interpolación

```
const sql = `SELECT * FROM usuarios
  WHERE username = '${usuario}'
  AND password = '${password}'`;
db.get(sql, (err, fila) => { /* respuesta */ });
```

Las comillas y operadores que recibe el formulario pasan a formar parte de la sentencia SQL. Al cerrar una cadena e introducir OR o UNION, la entrada altera la selección de filas. El programa concede acceso cuando db.get devuelve una fila, aunque las credenciales no sean válidas.

## 2.2 Hallazgos adicionales

La contraseña se guardaba en texto plano. La copia corregida utiliza scrypt con salt aleatorio.

El endpoint parametrizado ya existía, pero compartía proceso con el endpoint vulnerable. En la entrega segura solo existe POST /login protegido.

La base original no tenía la restricción UNIQUE que declaraba el código. CREATE TABLE IF NOT EXISTS no modificaba esa tabla existente; las bases nuevas sí incluyen username único.

La ruta relativa ./usuarios.db dependía del directorio de ejecución. Se cambió por una ruta basada en __dirname.

## 2.3 Errores y dependencias

El package.json inspeccionado era JSON válido; no se reprodujo ERR_INVALID_PACKAGE_CONFIG. No declaraba Express ni SQLite3 y ambos se resolvían desde una carpeta externa al proyecto. La entrega segura declara las dependencias y conserva package-lock.json.

Una comilla aislada produjo HTTP 400 con “Error SQL”. No se observó una fuga del mensaje detallado de SQLite, ni extracción de datos mediante errores.

# 3. Payloads que funcionaron

### 03 - Tautología

```
Usuario: nadie
Password: ' OR '1'='1
```

La comilla inicial cierra el valor de password. La consulta termina con OR '1'='1', una condición verdadera. Debido a la precedencia de AND sobre OR, basta esa condición para devolver una fila.

### 04 y 05 - OR y comentario

```
Usuario: admin
Password: ' OR 1=1 --
```

OR 1=1 vuelve verdadera la condición; -- comenta la comilla final en esa misma línea. Ambos casos usan la misma entrada: no son dos payloads diferentes.

### 07 - UNION

```
Usuario: nadie
Password: ' UNION SELECT 999,'demo','demo' --
```

UNION agrega una fila sintética con tres columnas, compatibles con id, username y password. Aunque no exista el usuario nadie, db.get recibe una fila y el programa concede acceso.

### 10 - Inferencia booleana

```
Usuario: admin
Password: ' OR (SELECT count(*) FROM usuarios)>0 --
```

La subconsulta comprueba que existe al menos una fila. Esa condición verdadera concede acceso. La variante <0 fue rechazada, mostrando una diferencia observable entre verdadero y falso.

Estos resultados demuestran bypass de la comprobación del login. La respuesta no revela columnas de la tabla, y la aplicación no crea una sesión: no se afirma extracción directa de cuentas ni acceso a recursos protegidos.

# 4. Corrección mediante parametrización

Se reemplazó la construcción de SQL con entradas interpoladas por una consulta fija y un arreglo de valores. La contraseña ya no participa en la sentencia SQL: se comprueba después mediante su hash.

```
db.get(
  `SELECT * FROM usuarios WHERE username = ?`,
  [usuario],
  (err, fila) => {
    if (err) {
      return res.status(500).json({mensaje: "Error interno"});
    }
    let valido = false;
    if (fila) {
      const [s, h] = fila.password.split(":");
      valido = timingSafeEqual(
        scryptSync(password, s, 64), Buffer.from(h, "hex")
      );
    }
    res.json({mensaje: valido
      ? "Acceso concedido" : "Acceso denegado"});
  }
);
```

## Por qué corrige SQL Injection

El signo ? representa un valor enlazado. El controlador recibe el texto SQL y [usuario] por separado: las comillas, OR, comentarios o UNION presentes en el usuario se tratan como datos. La contraseña solo llega a scrypt; no puede modificar la consulta.

## Medidas complementarias

Se validan tipos string y un máximo de 200 caracteres; la inserción inicial también usa parámetros. El formulario protegido oculta la contraseña, el servidor solo escucha en loopback y las bases quedan excluidas de Git.

La parametrización es la corrección de SQL Injection. El hash protege el almacenamiento de contraseñas y cumple una función diferente; la validación de longitud tampoco sustituye los parámetros.

# 5. Verificación de la corrección

Se aplicaron las mismas entradas a ambos formularios. Los resultados visibles coincidieron con los registros HTTP y con las expectativas de la matriz.

| ID | Caso | Vulnerable | Corregida |
|---|---|---|---|
| 01 | Credenciales correctas | Concedido | Concedido |
| 02 | Contraseña incorrecta | Denegado | Denegado |
| 03 | Tautología | Concedido | Denegado |
| 04 | OR y comentario | Concedido | Denegado |
| 05 | Condición verdadera | Concedido | Denegado |
| 06 | Condición falsa | Denegado | Denegado |
| 07 | UNION sintético | Concedido | Denegado |
| 08 | Comilla aislada | Error SQL | Denegado |
| 09 | Comentario en usuario | Denegado | Denegado |
| 10 | Inferencia verdadera | Concedido | Denegado |
| 11 | Inferencia falsa | Denegado | Denegado |

## Resultado negativo importante

El usuario `admin' --` con contraseña incorrecta no concedió acceso. En este código, AND password está en la siguiente línea; el comentario termina antes de esa condición. Por eso no se trasladan al reporte resultados de aplicaciones con consultas distintas.

## Regresión de la entrega segura

La carpeta P05-SQL-INJECTION se instaló con npm ci y se comprobó con npm test. Sus 11 casos pasaron: el acceso legítimo se conserva y todas las entradas de inyección se rechazan. Las capturas del anexo corresponden a las versiones del laboratorio, y el PR contiene la copia segura verificada.

No se realizaron pruebas temporales, canales externos ni operaciones destructivas. No se presenta esta matriz como una auditoría exhaustiva de una aplicación web.

# 6. Conclusiones y Pull Request

La causa del bypass fue interpretar la entrada del formulario como parte del SQL. Las cuatro entradas funcionales modificaron la selección de filas; la consulta parametrizada evitó esa modificación y la verificación por hash rechazó las contraseñas de ataque.

Repetir controles válidos e inválidos permitió comprobar que la corrección no consistió en denegar todos los intentos. También se documentaron resultados negativos y las limitaciones del login, sin atribuirle extracción de datos ni sesiones que no implementa.

Como trabajo futuro, un sistema de autenticación completo requeriría manejo de sesiones, autorización y protección frente a intentos repetidos. Esta entrega se limita a la práctica local y a la corrección de SQL Injection.

## Pull Request de la versión segura

https://github.com/Kyaajjs/66575_Zarate/pull/1

Repositorio: Kyaajjs/66575_Zarate. Rama: codex/practica-sqli-segura. Destino: main. Estado comprobado: abierto, sin fusionar. Incluye servidor seguro, formulario, dependencias, pruebas y evidencias.

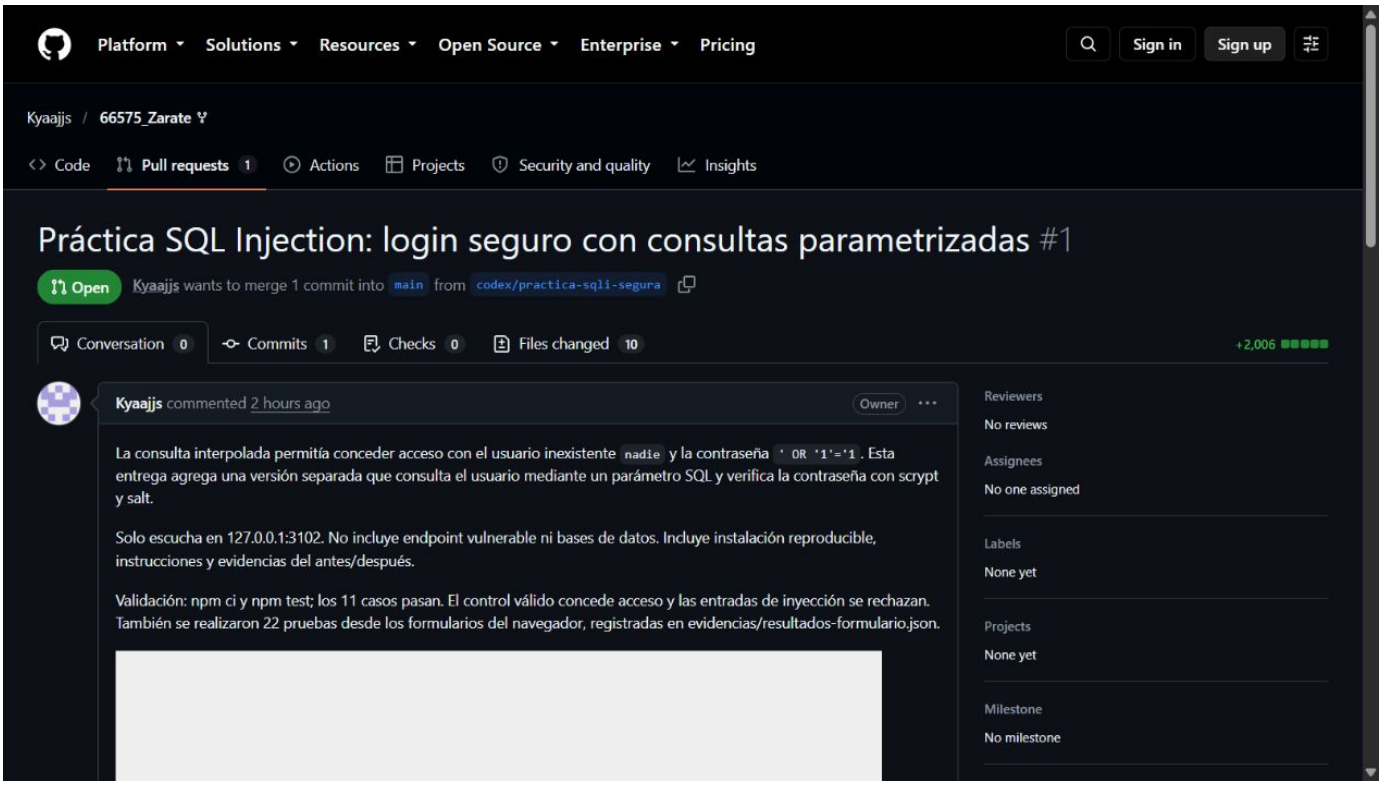



Figura 1. Captura real del PR #1 al momento de su creación. El enlace permite consultar su estado actualizado.

# Anexo | Caso 01

## Control válido

```
Usuario: admin
Password: 12345
```

Control positivo: confirma que admin / 12345 mantiene el acceso en ambas versiones.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso concedido

Figuras 2 y 3. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 01-vulnerable.png / 01-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:10:39.376Z / 2026-10-27T00:11:36.374Z

# Anexo | Caso 02

## Control inválido

```
Usuario: admin
Password: incorrecta
```

Control negativo: una contraseña incorrecta debe rechazarse antes y después.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso denegado
Resultado después: Acceso denegado

Figuras 4 y 5. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 02-vulnerable.png / 02-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:10:51.061Z / 2026-10-27T00:11:40.229Z

# Anexo | Caso 03

## Tautología en contraseña

```
Usuario: nadie
Password: ' OR '1'='1
```

La tautología devuelve una fila sin credenciales válidas; en la versión corregida se rechaza.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso denegado

Figuras 6 y 7. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 03-vulnerable.png / 03-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:10:57.268Z / 2026-10-27T00:11:43.855Z

# Anexo | Caso 04

## Comentario en contraseña

```
Usuario: admin
Password: ' OR 1=1 --
```

El OR verdadero concede acceso; el comentario neutraliza la comilla restante en esa línea.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso denegado

Figuras 8 y 9. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 04-vulnerable.png / 04-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:02.158Z / 2026-10-27T00:11:46.676Z

# Anexo | Caso 05

## Condición verdadera

```
Usuario: admin
Password: ' OR 1=1 --
```

Misma entrada que el caso 04, conservada como condición verdadera del par 05/06.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso denegado

Figuras 10 y 11. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 05-vulnerable.png / 05-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:06.068Z / 2026-10-27T00:11:49.502Z

# Anexo | Caso 06

## Condición falsa

```
Usuario: admin
Password: ' OR 1=2 --
```

La variante OR 1=2 no vuelve verdadera la condición. Es el control falso del caso 05.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso denegado
Resultado después: Acceso denegado

Figuras 12 y 13. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 06-vulnerable.png / 06-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:11.241Z / 2026-10-27T00:11:52.909Z

# Anexo | Caso 07

## UNION de fila sintética

```
Usuario: nadie
Password: ' UNION SELECT 999,'demo','demo' --
```

UNION agrega una fila sintética de tres columnas. Se observa bypass, no extracción de columnas.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso denegado

Figuras 14 y 15. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 07-vulnerable.png / 07-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:15.742Z / 2026-10-27T00:11:58.522Z

# Anexo | Caso 08

## Comilla sin cerrar

```
Usuario: admin
Password: '
```

La comilla aislada rompe la sintaxis vulnerable. La respuesta es genérica: no expone un error detallado del motor.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Error SQL
Resultado después: Acceso denegado

Figuras 16 y 17. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 08-vulnerable.png / 08-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:20.416Z / 2026-10-27T00:12:01.684Z

# Anexo | Caso 09

## Comentario en usuario (salto de línea)

```
Usuario: admin' --
Password: incorrecta
```

El comentario en usuario termina en el salto de línea; la comparación de password sigue activa.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso denegado
Resultado después: Acceso denegado

Figuras 18 y 19. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 09-vulnerable.png / 09-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:25.141Z / 2026-10-27T00:12:04.948Z

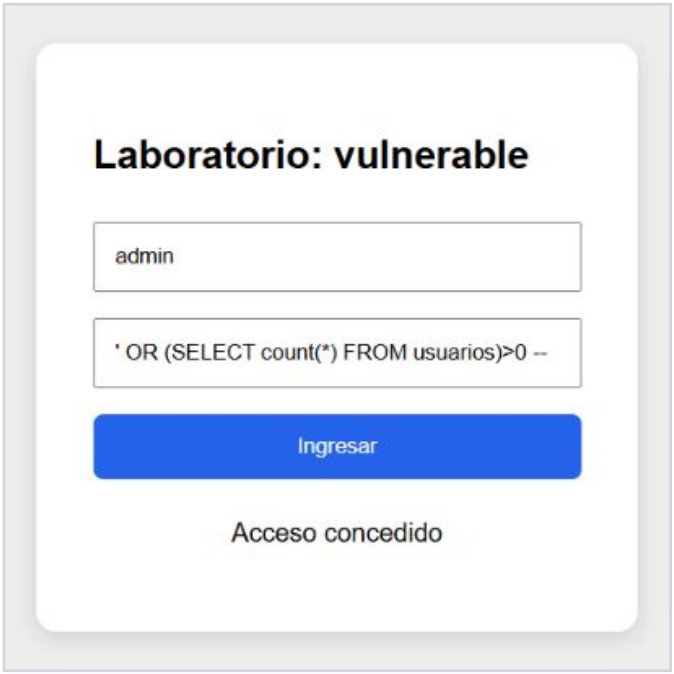
# Anexo | Caso 10

## Inferencia booleana verdadera

```
Usuario: admin
Password: ' OR (SELECT count(*) FROM usuarios)>0 --
```

La subconsulta count(*)>0 es verdadera y genera una señal de inferencia booleana.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso concedido
Resultado después: Acceso denegado

Figuras 20 y 21. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 10-vulnerable.png / 10-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:29.402Z / 2026-10-27T00:12:09.333Z

# Anexo | Caso 11

## Inferencia booleana falsa

```
Usuario: admin
Password: ' OR (SELECT count(*) FROM usuarios)<0 --
```

La subconsulta count(*)<0 es falsa. Junto al caso 10 muestra el contraste verdadero/falso.

**ANTES | Vulnerable**




**DESPUÉS | Corregida**




Resultado antes: Acceso denegado
Resultado después: Acceso denegado

Figuras 22 y 23. Capturas del formulario real tras pulsar Ingresar. Se muestra el área central de cada captura para facilitar la lectura; los PNG completos se conservan como evidencia.

En la versión corregida la contraseña aparece oculta. Su contenido exacto es el indicado arriba y el registrado en resultados-formulario.json. Las entradas largas pueden no verse completas dentro del campo.

Archivos: 11-vulnerable.png / 11-corregida.png
Fecha de ejecución (UTC): 2026-10-27T00:11:32.815Z / 2026-10-27T00:12:14.601Z